

100 Days of Azure DevOps

Day 48 Handout — Remote state in Azure Storage with a blob lease lock. Two applies at once is a tug-of-war with reality

5 - Infrastructure as Code · Personal lab only · Educational content · Not a sales pitch

Architecture A — backend azurerm + a module boundary

Piece	Config	Why
State RG + account	rg-tfstate / st<unique> / container tfstate	Holds the memory. Note it costs (cheap LRS)
backend "azurerm"	resource_group_name, storage_account_name, container_name, key	key = lab.tfstate (path in the container)
Lock	Blob lease while apply/plan (depending on version)	Second apply waits or errors — do not force-unlock
Module	modules/storage with inputs	Reuse without three nearly identical main.tf files
Migrate	Add backend; terraform init — migrate state? yes	Do not copy tfstate like a raccoon

Architecture B — destroy order (this is the lab)

Order	Do	Never
1	destroy workload (the module stack)	Delete the storage account from Portal first
2	Confirm Azure empty for that stack	Leave resources with remote state still pointing at them
3	Only then delete rg-tfstate if you are done with Phase 5	Destroy state while resources exist = amnesia on purpose
Lock stuck	Wait / find the other apply	terraform force-unlock because you are impatient

Architecture C — pitfalls

Pitfall	What you see	Fix
Two local states	Duplicate RGs; neither destroy is complete	One backend, one key, locking on
State account in the workload RG	You cannot destroy the RG without destroying memory	Separate rg-tfstate from rg-day48-lab
force-unlock	Two applies interleaved; duplicate names	Treat unlock as an incident, not a shortcut
Module copy-paste	Three storage blocks drifting	module "st" { source = "../modules/storage" }

One-liner to remember

Remote state with locking stops two applies from playing tug-of-war with production. Destroy the workload before you destroy the memory.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal subscription. Separate rg-tfstate from the workload RG. Note storage cost. No force-unlock.

- 1 Create rg-tfstate + a Standard_LRS storage account + blob container tfstate (CLI). This account is the backend, not the app.
- 2 Add terraform { backend "azurerm" { ... key = "lab.tfstate" } }. terraform init and accept state migration from local.
- 3 Extract storage (or RG+storage) into modules/storage. Call it from main. terraform plan/apply through the backend once.

- 4 In Azure Storage, confirm the blob lab.tfstate exists. Do not download it to email.
- 5 terraform destroy the workload. Leave rg-tfstate until you are sure. If you also delete state storage, do it last.
- 6 If a lock exists, wait. Do not force-unlock. Write that rule in docs/tf-state-day48.md.

backend azurerm (partial config; fill unique names)

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "rg-tfstate"  
    storage_account_name = "sttfstate<unique>"  
    container_name      = "tfstate"  
    key                  = "lab.tfstate"  
  }  
}  
  
# az group create -n rg-tfstate -l centralindia  
# az storage account create -g rg-tfstate -n sttfstate<unique> --sku Standard_LRS  
# az storage container create --account-name sttfstate<unique> -n tfstate  
# terraform init # migrate local state when prompted  
# terraform plan && terraform apply  
# terraform destroy # workload first; state account last
```

Done checklist

- State lives in Azure Storage, not only on the laptop
- Workload RG ≠ state RG
- Did not force-unlock
- Workload destroyed; state account not deleted first

Tomorrow — Day 49

IaC in pipelines — plan on a clean agent, publish the plan, apply only with a brake.